PROBLEM STATEMENT Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. They aim to build the operating system for commerce, through a combination of world-class infrastructure, logistics operations of the highest quality, and cutting-edge engineering and technology capabilities.

The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors.

IMPORTING LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.preprocessing import StandardScaler,MinMaxScaler
```

UPLOADING DATASET

```
df=pd.read_csv("delhivery_data.txt")
```

```
df.head()
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	c
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	04:4
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	04:4
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	04:4
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	04:4
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	04:4

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   data                            144867 non-null  object
 1   trip_creation_time              144867 non-null  object
 2   route_schedule_uuid             144867 non-null  object
 3   route_type                      144867 non-null  object
 4   trip_uuid                       144867 non-null  object
 5   source_center                   144867 non-null  object
 6   source_name                     144574 non-null  object
 7   destination_center              144867 non-null  object
 8   destination_name                144606 non-null  object
 9   od_start_time                   144867 non-null  object
 10  od_end_time                     144867 non-null  object
 11  start_scan_to_end_scan          144867 non-null  float64
 12  is_cutoff                       144867 non-null  bool
 13  cutoff_factor                   144867 non-null  int64
 14  cutoff_timestamp                144867 non-null  object
 15  actual_distance_to_destination  144867 non-null  float64
 16  actual_time                     144867 non-null  float64
 17  osrm_time                       144867 non-null  float64
 18  osrm_distance                   144867 non-null  float64
 19  factor                          144867 non-null  float64
 20  segment_actual_time             144867 non-null  float64
 21  segment_osrm_time               144867 non-null  float64
 22  segment_osrm_distance           144867 non-null  float64
 23  segment_factor                  144867 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB
```

```
df.describe()
```

	start_scan_to_end_scan	cutoff_factor	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	factor	segment_actual_time	segment_osrm_time	segm
count	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	
mean	961.262986	232.926567	234.073372	416.927527	213.868272	284.771297	2.120107	36.196111	18.507548	
std	1037.012769	344.755577	344.990009	598.103621	308.011085	421.119294	1.715421	53.571158	14.775960	
min	20.000000	9.000000	9.000045	9.000000	6.000000	9.008200	0.144000	-244.000000	0.000000	
25%	161.000000	22.000000	23.355874	51.000000	27.000000	29.914700	1.604264	20.000000	11.000000	
50%	449.000000	66.000000	66.126571	132.000000	64.000000	78.525800	1.857143	29.000000	17.000000	
75%	1634.000000	286.000000	286.708875	513.000000	257.000000	343.193250	2.213483	40.000000	22.000000	
max	7898.000000	1927.000000	1927.447705	4532.000000	1686.000000	2326.199100	77.387097	3051.000000	1611.000000	

```
missing=df.isnull().sum()
```

```
print(missing)
```

```
0
```

DATASET HAS NO NULL VALUES

CONVERT DATETIME

```
df['trip_creation_time'] = pd.to_datetime(df['trip_creation_time'])
df['od_start_time']      = pd.to_datetime(df['od_start_time'])
df['od_end_time']        = pd.to_datetime(df['od_end_time'])

print("Date columns converted!")
print(df[['trip_creation_time', 'od_start_time', 'od_end_time']].dtypes)
```

```
Date columns converted!
trip_creation_time    datetime64[ns]
od_start_time         datetime64[ns]
od_end_time           datetime64[ns]
dtype: object
```

CREATE SEGMENTKEY

```
df['segment_key'] = (
    df['trip_uuid'] + '|' +
    df['source_center'] + '|' +
    df['destination_center']
)

print("Segment key created!")
print("Example:", df['segment_key'][0])
```

```
Segment key created!
Example: trip-153741093647649320|IND388121AAA|IND388620AAB
```

ADD RUNNING TOTAL COLUMNS

```
df['segment_actual_time_sum']   = df.groupby('segment_key')['segment_actual_time'].cumsum()
df['segment_osrm_time_sum']     = df.groupby('segment_key')['segment_osrm_time'].cumsum()
df['segment_osrm_distance_sum'] = df.groupby('segment_key')['segment_osrm_distance'].cumsum()

print("Running total columns created!")
df[['segment_key', 'segment_actual_time_sum']].head()
```

```
Running total columns created!
```

	segment_key	segment_actual_time_sum
0	trip-153741093647649320|IND388121AAA|IND388620AAB	14.0
1	trip-153741093647649320|IND388121AAA|IND388620AAB	24.0
2	trip-153741093647649320|IND388121AAA|IND388620AAB	40.0
3	trip-153741093647649320|IND388121AAA|IND388620AAB	61.0
4	trip-153741093647649320|IND388121AAA|IND388620AAB	67.0

COLLAPSE ROW INTO ONE ROW SEGMENT

```
rules = {
    'data':                           'first',
    'trip_creation_time':             'first',
    'route_type':                     'first',
    'trip_uuid':                      'first',
    'source_center':                  'first',
    'source_name':                    'first',
    'destination_center':             'last',
    'destination_name':               'last',
    'od_start_time':                  'first',
    'od_end_time':                    'last',
    'actual_distance_to_destination': 'last',
    'actual_time':                    'last',
    'osrm_time':                      'last',
    'osrm_distance':                  'last',
    'segment_actual_time':            'sum',
    'segment_osrm_time':              'sum',
    'segment_osrm_distance':          'sum',
    'segment_actual_time_sum':        'last',
    'segment_osrm_time_sum':          'last',
    'segment_osrm_distance_sum':      'last',
}
segment = df.groupby('segment_key').agg(rules).reset_index()

print("Done! Each segment is now one row.")
print("Shape:", segment.shape)
segment.head()
```

```
Done! Each segment is now one row.
Shape: (26368, 21)
```

	segment_key	data	trip_creation_time	route_type	trip_uuid	source_center	source_name	destination_center	destina
0	trip-153671041653548748|IND209304AAA|IND000000ACB	training	2018-09-12 00:00:16.535741	FTL	trip-153671041653548748	IND209304AAA	Kanpur_Central_H_6 (Uttar Pradesh)	IND000000ACB	Gurgaon_B
1	trip-153671041653548748|IND462022AAA|IND209304AAA	training	2018-09-12 00:00:16.535741	FTL	trip-153671041653548748	IND462022AAA	Bhopal_Trnsport_H (Madhya Pradesh)	IND209304AAA	Kanpur_C (Utta
2	trip-153671042288605164|IND561203AAB|IND562101AAA	training	2018-09-12 00:00:22.886430	Carting	trip-153671042288605164	IND561203AAB	Doddablpur_ChikaDPP_D (Karnataka)	IND562101AAA	Chikblapur_ (
3	trip-153671042288605164|IND572101AAA|IND561203AAB	training	2018-09-12 00:00:22.886430	Carting	trip-153671042288605164	IND572101AAA	Tumkur_Veersagr_I (Karnataka)	IND561203AAB	Doddablpur_Cl (
4	trip-153671043369099517|IND000000ACB|IND160002AAC	training	2018-09-12 00:00:33.691250	FTL	trip-153671043369099517	IND000000ACB	Gurgaon_Bilaspur_HB (Haryana)	IND160002AAC	Chandigarh_Me

Double-click (or enter) to edit

CALCULATE TRIP DURATION IN HOURS

```
segment['trip_duration_hours'] = (
    segment['od_end_time'] - segment['od_start_time']
).dt.total_seconds() / 3600

print("Trip duration column created!")
print(segment['trip_duration_hours'].describe())
```

```
Trip duration column created!
count    26368.000000
mean         4.980538
std          7.344894
min          0.345047
25%          1.517248
50%          2.541975
75%          5.118318
max        131.642533
Name: trip_duration_hours, dtype: float64
```

EXTRACT STATE NAME FROM LOCATION NAME

```
segment['destination_state'] = segment['destination_name'].str.extract(r'\(([^)]+)\)')
segment['source_state']      = segment['source_name'].str.extract(r'\(([^)]+)\)')
segment['destination_city'] = segment['destination_name'].str.split('_').str[0]
segment['source_city']      = segment['source_name'].str.split('_').str[0]

print("State and city columns created!")
segment[['source_name', 'source_city', 'source_state']].head()
```

```
State and city columns created!
```

	source_name	source_city	source_state
0	Kanpur_Central_H_6 (Uttar Pradesh)	Kanpur	Uttar Pradesh
1	Bhopal_Trnsport_H (Madhya Pradesh)	Bhopal	Madhya Pradesh
2	Doddablpur_ChikaDPP_D (Karnataka)	Doddablpur	Karnataka
3	Tumkur_Veersagr_I (Karnataka)	Tumkur	Karnataka
4	Gurgaon_Bilaspur_HB (Haryana)	Gurgaon	Haryana

EXTRACT MONTH,DAY,HOUR FROM TRIP TIME

```
segment['trip_month']   = segment['trip_creation_time'].dt.month
segment['trip_day']     = segment['trip_creation_time'].dt.day
segment['trip_hour']    = segment['trip_creation_time'].dt.hour
segment['trip_weekday'] = segment['trip_creation_time'].dt.dayofweek
print("Date feature columns created!")
segment[['trip_month', 'trip_day', 'trip_hour', 'trip_weekday']].head()
```

```
Date feature columns created!
```

	trip_month	trip_day	trip_hour	trip_weekday
0	9	12	0	2
1	9	12	0	2
2	9	12	0	2
3	9	12	0	2
4	9	12	0	2

COLLAPSE ONE ROW PER TRIP

```
trip_rules = {
    'route_type':                     'first',
    'source_name':                    'first',
    'source_city':                    'first',
    'source_state':                   'first',
    'destination_name':               'last',
    'destination_city':               'last',
    'destination_state':              'last',
    'actual_distance_to_destination': 'last',
    'actual_time':                    'sum',
    'osrm_time':                      'sum',
    'osrm_distance':                  'sum',
    'segment_actual_time':            'sum',
    'segment_osrm_time':              'sum',
    'segment_osrm_distance':          'sum',
    'trip_duration_hours':            'sum',
    'trip_month':                     'first',
    'trip_weekday':                   'first',
    'trip_hour':                      'first',
}

trip = segment.groupby('trip_uuid').agg(trip_rules).reset_index()

print("Trip-level table created!")
print("Shape:", trip.shape)
trip.head()
```

```
Trip-level table created!
Shape: (14817, 19)
```

	trip_uuid	route_type	source_name	source_city	source_state	destination_name	destination_city	destination_state	actual_distance_to_destination	act
0	trip-153671041653548748	FTL	Kanpur_Central_H_6 (Uttar Pradesh)	Kanpur	Uttar Pradesh	Kanpur_Central_H_6 (Uttar Pradesh)	Kanpur	Uttar Pradesh	440.973689	
1	trip-153671042288605164	Carting	Doddablpur_ChikaDPP_D (Karnataka)	Doddablpur	Karnataka	Doddablpur_ChikaDPP_D (Karnataka)	Doddablpur	Karnataka	48.542890	
2	trip-153671043369099517	FTL	Gurgaon_Bilaspur_HB (Haryana)	Gurgaon	Haryana	Gurgaon_Bilaspur_HB (Haryana)	Gurgaon	Haryana	1689.964663	
3	trip-153671046011330457	Carting	Mumbai Hub (Maharashtra)	Mumbai Hub (Maharashtra)	Maharashtra	Mumbai_MiraRd_IP (Maharashtra)	Mumbai	Maharashtra	17.175274	
4	trip-153671052974046625	FTL	Bellary_Dc (Karnataka)	Bellary	Karnataka	Sandur_WrdN1DPP_D (Karnataka)	Sandur	Karnataka	26.600536	

Next steps: Generate code with `trip` New interactive sheet

FIND OUTLIER WITH BOXPLOT

```
cols_to_check = ['actual_time', 'osrm_time', 'actual_distance_to_destination', 'trip_duration_hours']
```

```
fig, axes = plt.subplots(1, 4, figsize=(18, 5))

for i, col in enumerate(cols_to_check):
    axes[i].boxplot(trip[col].dropna())
    axes[i].set_title(col)

plt.suptitle("Boxplots — Dots are Outliers", fontsize=13)
plt.tight_layout()
plt.show()
```


REMOVE OUTLIERS

```
cols_to_clean = ['actual_time', 'osrm_time', 'actual_distance_to_destination', 'trip_duration_hours']

trip_clean = trip.copy()   # make a copy so original 'trip' stays safe

for col in cols_to_clean:
    Q1  = trip_clean[col].quantile(0.25)
    Q3  = trip_clean[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    before = len(trip_clean)
    trip_clean = trip_clean[
        (trip_clean[col] >= lower_limit) &
        (trip_clean[col] <= upper_limit)
    ]
    print(f"{col}: removed {before - len(trip_clean)} outlier rows")

print("\nRows before:", len(trip))
print("Rows after:", len(trip_clean))
```

```
actual_time: removed 1643 outlier rows
osrm_time: removed 740 outlier rows
actual_distance_to_destination: removed 1103 outlier rows
trip_duration_hours: removed 810 outlier rows

Rows before: 14817
Rows after: 10521
```

ONE HOT ENCODING

```
trip_encoded = pd.get_dummies(trip_clean, columns=['route_type'], drop_first=True)

print("Encoding done!")
print("New columns:", [c for c in trip_encoded.columns if 'route_type' in c])
trip_encoded.head()
```

```
Encoding done!
New columns: ['route_type_FTL']
```

	trip_uuid	source_name	source_city	source_state	destination_name	destination_city	destination_state	actual_distance_to_destination	actual_time	os
1	trip-153671042288605164	Doddablpur_ChikaDPP_D (Karnataka)	Doddablpur	Karnataka	Doddablpur_ChikaDPP_D (Karnataka)	Doddablpur	Karnataka	48.542890	143.0	
3	trip-153671046011330457	Mumbai Hub (Maharashtra)	Mumbai Hub (Maharashtra)	Maharashtra	Mumbai_MiraRd_IP (Maharashtra)	Mumbai	Maharashtra	17.175274	59.0	
5	trip-153671055416136166	Chennai_Poonamallee (Tamil Nadu)	Chennai	Tamil Nadu	Chennai_Poonamallee (Tamil Nadu)	Chennai	Tamil Nadu	9.271519	61.0	
6	trip-153671066201138152	Chennai_Chrompet_DPC (Tamil Nadu)	Chennai	Tamil Nadu	Chennai_Vandalur_Dc (Tamil Nadu)	Chennai	Tamil Nadu	9.100510	24.0	
7	trip-153671066826362165	HBR Layout PC (Karnataka)	HBR Layout PC (Karnataka)	Karnataka	HBR Layout PC (Karnataka)	HBR Layout PC (Karnataka)	Karnataka	12.352948	64.0	

Next steps: Generate code with `trip_encoded` New interactive sheet

SCALE(NORMALIZE) THE NUMBERS

```
cols_to_scale = ['actual_time', 'osrm_time', 'actual_distance_to_destination', 'trip_duration_hours']
cols_to_scale = [c for c in cols_to_scale if c in trip_encoded.columns]

scaler = MinMaxScaler()
trip_encoded[cols_to_scale] = scaler.fit_transform(trip_encoded[cols_to_scale])

print("Scaling done! Values are now between 0 and 1:")
trip_encoded[cols_to_scale].head()
```

Scaling done! Values are now between 0 and 1:

	actual_time	osrm_time	actual_distance_to_destination	trip_duration_hours
1	0.201201	0.239216	0.476755	0.232522
3	0.075075	0.031373	0.098543	0.113259
5	0.078078	0.062745	0.003244	0.245572
6	0.022523	0.023529	0.001182	0.109599
7	0.082583	0.105882	0.040398	0.181396

HYPOTHESIS TESTING

```
pairs = [
    ('actual_time',   'osrm_time',             'Actual Time vs OSRM Time'),
    ('actual_time',   'segment_actual_time',   'Actual Time vs Segment Actual Time'),
    ('osrm_distance', 'segment_osrm_distance', 'OSRM Distance vs Segment OSRM Distance'),
    ('osrm_time',     'segment_osrm_time',     'OSRM Time vs Segment OSRM Time'),
]

print("=" * 60)
for col1, col2, label in pairs:
    if col1 in trip.columns and col2 in trip.columns:
        data = trip[[col1, col2]].dropna().sample(min(5000, len(trip)), random_state=42)
        stat, p = stats.wilcoxon(data[col1], data[col2])
        print(f"\n🔬 {label}")
        print(f"   p-value: {p:.6f}")
        if p < 0.05:
            print("   ✅ SIGNIFICANTLY DIFFERENT — columns measure differently")
        else:
            print("   ⚖️  NOT significantly different — columns are similar")
```

```
============================================================

🔬 Actual Time vs OSRM Time
   p-value: 0.000000
   ✅ SIGNIFICANTLY DIFFERENT — columns measure differently

🔬 Actual Time vs Segment Actual Time
   p-value: 0.000000
   ✅ SIGNIFICANTLY DIFFERENT — columns measure differently

🔬 OSRM Distance vs Segment OSRM Distance
   p-value: 0.000000
   ✅ SIGNIFICANTLY DIFFERENT — columns measure differently

🔬 OSRM Time vs Segment OSRM Time
   p-value: 0.000000
   ✅ SIGNIFICANTLY DIFFERENT — columns measure differently
```

BUSINESS INSIGHT

1.There are two types of trips:

FTL (Full Truck Load) — One big truck goes directly from source to destination. Faster and used for long distances. Carting — Small vehicles used for short distances, like last-mile delivery within a city 2.Looking at the source names in the data, most orders originate from Maharashtra, Gujarat, Karnataka, and Tamil Nadu.Delhivery should ensure maximum warehouse capacity and staff availability in these three states. 3.Within-state deliveries dominate. Delhivery should focus on optimizing local city routes, not just long-distance highways. 4. OSRM underestimates actual delivery time. This could be because OSRM does not account for:

Loading and unloading time at warehouses Traffic jams during peak hours Waiting time at toll booths or checkpoints **bold text**

BUSINESS RECOMMENDATION

Close

```
1. Expand capacity in Maharashtra, Gujarat, and Karnataka
These three states generate the majority of orders. More sorting centers and
delivery hubs here will reduce delays.
2. Fix OSRM time predictions
Add real-world delay factors (traffic, loading time) to the routing engine.
Don't rely only on theoretical shortest-path calculations.
3. Investigate the time gap between segments
Find out why total actual time doesn't match the sum of individual segment
times. This is a data quality issue that could lead to wrong forecasting.
4. Prepare for festive season spikes
September data shows pre-festive activity. Delhivery should start hiring
temporary staff and adding vehicles from August itself.
5. Optimize within-state corridors
Since most deliveries are within the same state, improving local city routes
(better road selection, smarter Carting assignments) will have the biggest
impact on delivery speed.
```

1. Expand capacity in Maharashtra, Gujarat, and Karnataka These three states generate the majority of orders. More sorting centers and delivery hubs here will reduce delays.
2. Fix OSRM time predictions Add real-world delay factors (traffic, loading time) to the routing engine. Don't rely only on theoretical shortest-path calculations.
3. Investigate the time gap between segments Find out why total actual time doesn't match the sum of individual segment times. This is a data quality issue that could lead to wrong forecasting.
4. Prepare for festive season spikes September data shows pre-festive activity. Delhivery should start hiring temporary staff and adding vehicles from August itself.
5. Optimize within-state corridors Since most deliveries are within the same state, improving local city routes (better road selection, smarter Carting assignments) will have the biggest impact on delivery speed.